

Code Snippet 1

```
void validate_checkout_request(input_form i)
{
    if (!valid_string(i.name)) {
        error_message("Invalid name");
    }

    if (!valid_string(i.book)) {
        error_message("Invalid book name");
    }

    if (!valid_month(i.date)) {
        error_message("Invalid month");
    }
}

int valid_month(date d)
{
    return d.month >= 1 && d.month <= 12;
}
```

通信内聚、数据耦合

Code Snippet 2

```
void validate_checkout_request(input_form i)
{
    if (!valid_string(i)) {
        error_message("Invalid name");
    }

    if (!valid_string(i)) {
        error_message("Invalid book name");
    }

    if (!valid_month(i)) {
        error_message("Invalid month");
    }
}

int valid_month(input_form i)
{
    return i.date.month >= 1 && i.date.month <= 12;
}
```

印记耦合

Code Snippet 3

```
void validate_checkout_request(input_form i)
{
    if (!valid(i.name, STRING)) {
        error_message("Invalid name");
    }

    if (!valid(i.book, STRING)) {
        error_message("Invalid book name");
    }

    if (!valid(i.date, DATE)) {
        error_message("Invalid month");
    }
}

int valid(string s, int type)
{
    switch (type) {
        case STRING:
            return strlen(s) < MAX_STRING_SIZE;
        case DATE:
            date d = parse_date(s);
            return d.month >= 1 && d.month <= 12;
    };
}
```

控制耦合

Code Snippet 4

```
string patron_name, book_name;
date checkout_date;

void validate_checkout_request(input_form i)
{
    patron_name = i.name;
    if (!valid_string()) {
        error_message("Invalid name");
    }

    book_name = i.book;
    if (!valid_string()) {
        error_message("Invalid book name");
    }

    checkout_date = i.date;
    if (!valid_month()) {
        error_message("Invalid month");
    }
}

int valid_month()
{
    return checkout_date.month >= 1 && checkout_date.month <= 12;
}
```

公共耦合

Code Snippet 5

```
void validate_checkout_request(input_form i)
{
    if (!valid_string(i.name)) {
        i.string = "Invalid name";
        error_message();
    }

    if (!valid_string(i.book)) {
        i.book = "Invalid book name";
        error_message();
    }

    valid_month(i.date);
}

void valid_month(date d)
{
    if (d.month < 1) {
        d.month = 1;
    }

    if (d.month > 12) {
        d.month = 12;
    }

    return 1;
}
```

内容耦合

Code Snippet 6 -- 重复耦合

```
void validate_checkout_request(input_form
i)
{
    int len = 0;
    boolean valid_string = false;

    len = i.name.length();
    char arr1[] = new char[len];
    for(char c: arr1){
        if c是小写字母 valid_string = true;
    }
    if (!valid_string) {
        error_message("Invalid name");
    }

    len = i.book.length();
    char arr2[] = new char[len];
    for(char c: arr2){
        if c是小写字母 valid_string = true;
    }
    if (!valid_string) {
        error_message("Invalid book name");
    }

    if (!valid_month(i.date)) {
        error_message("Invalid month");
    }
}
```

Public class **Rous**{

```
public static int findPattern( String text, String pattern)
{ ... }
public static int average( Vector numbers )
{ ... }
public static OutputStream openFile( String fileName )
{ ... }
}
```

偶然内聚

```

public void sample( String flag ) {
    switch ( flag ){
        case ON:
            .....
            break;
        case OFF:
            .....
            break;
        case CLOSE:
            .....
            break;
    }
}

```

逻辑内聚

```

Public class foo {
    Private string name;
    Private int size;
    //constructor
    public void foo(){

        this.name = "Not Set";
        this.size = 12;
    }
    //destructor
    Public void ~foo() {
        delete[] name;
        delete size;
    }
}

```

时间内聚

```

void MonthEnd()
{
    Report ExR = InitExpenseReport();
    Report rr = InitRevenueReport();
    Report EmpR = InitEmployeeReport();

    EmpR.Init();
    rr.Init();
    ExR.SetEmployees(true);

    if (ExR.GetReportParams())
        EmpR.GetReportParams();

    SendToPrinter(rr);
    SendToPrinter(ExR);
    SendToPrinter(EmpR);
}

```

过程内聚

```

Public class Calculate {
    Public int product;
    public void product(int a, int b){
        product = a*b;
        ....
        save(product);
        ....
    }
    Public void save(int product) {
        \\ code to store value into database
        ...
    }
}

```

通信内聚

```
public int commission(int sale, long percentage){  
    int com;  
    //calculate commission  
    return com;  
}
```

功能内聚

```
Public interface Addressee{  
    .....  
    Public abstract String getName ();  
    Public abstract String getAddress ();  
    ....  
}  
  
Public class Employee implements Addressee {....}
```

信息内聚

```
public class Vector3D{
    public int x, y, z;
    ...
}
public class Arch {
    private Vector3D baseline;
    ...
    void slant(int newY){
        baseline.x = 10;
        baseline.y = 13;
    }
}
```

内容耦合

```
public routineX(String command) {
    if (command.equals("drawCircle")){
        drawCircle();
    }
    else{
        drawRectangle();
    }
}
```

控制耦合

```

public class Employee{
    public String name, emailID.
    ...
}

public class Emailer{
    public void sendEmail(Employee e, String text)
    {...}
    ...
}

```

印记耦合

```

Public class Receiver {
    public void message( MyType X ){
        ...
        X.doSomethingForMe( Object data );
        ...
    }
}

```

数据耦合

```

int x;

public class myValue{
    public void addValue(int a){
        x = x+ a;
    }
    public void subtractValue(int a){
        x = x- a;
    }
}

```

公共耦合

